

Assignment 20.3

Htno:2303a51291

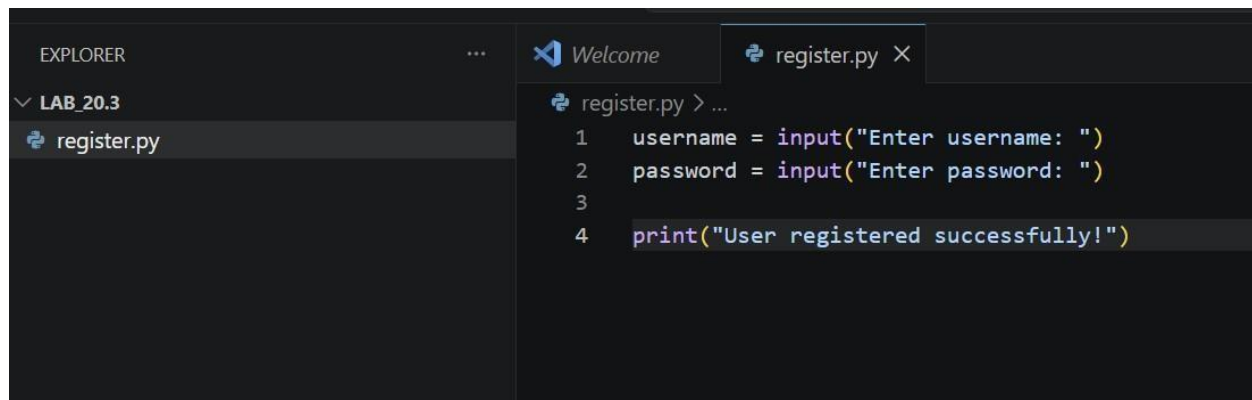
Btno:05

Task1: Password Strength & Validation Check

Prompt:Generate Python user registration program

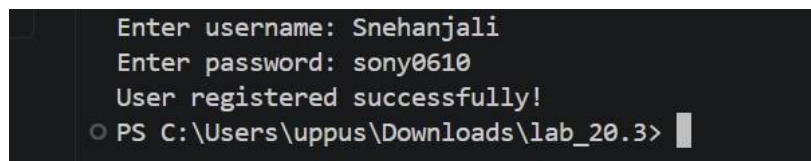
Code:

AI-Generated Code:

A screenshot of a code editor interface. On the left, the 'EXPLORER' pane shows a folder named 'LAB_20.3' containing a file 'register.py'. The main editor area has two tabs: 'Welcome' and 'register.py'. The 'register.py' tab is active, displaying the following Python code:

```
1 username = input("Enter username: ")
2 password = input("Enter password: ")
3
4 print("User registered successfully!")
```

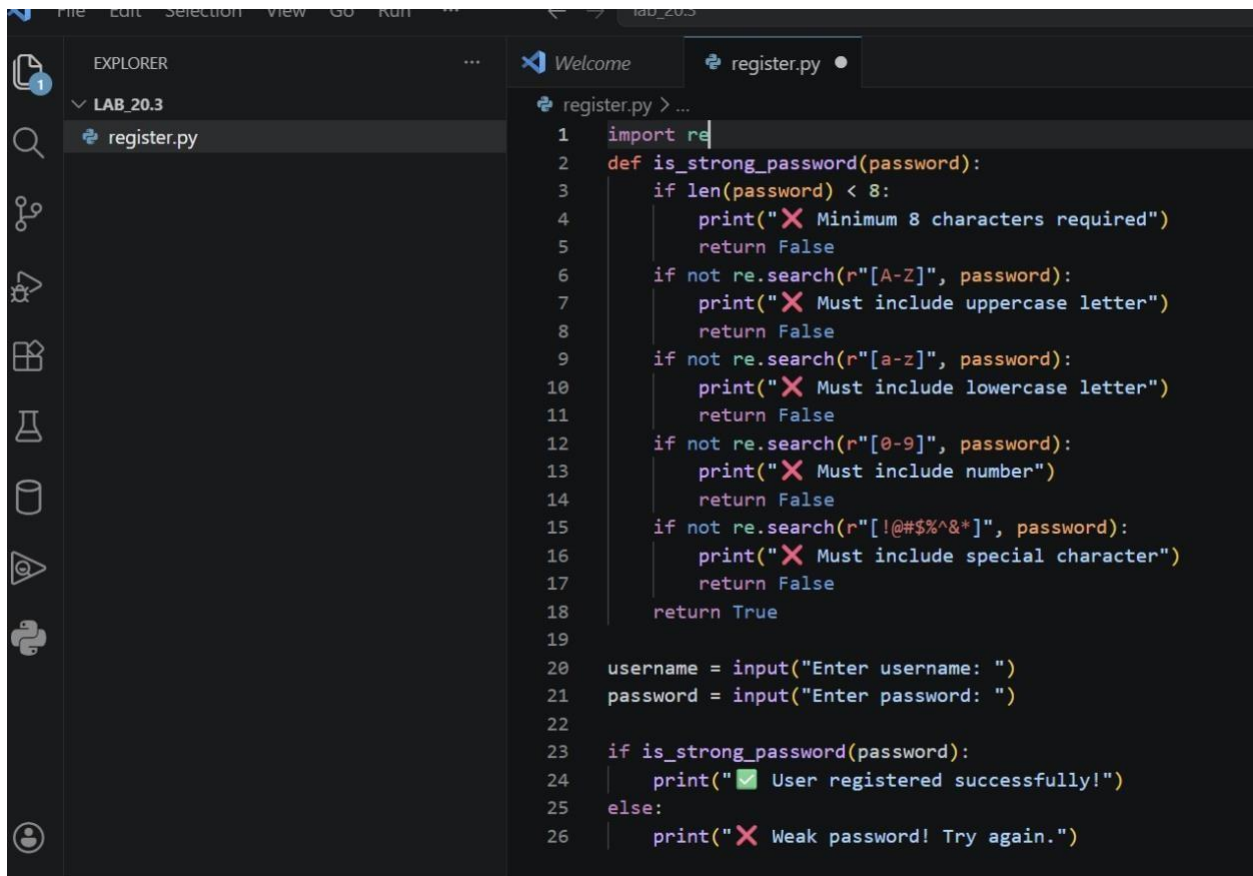
Output:

A screenshot of a terminal window. It shows the execution of the Python program. The prompts and user input are as follows:

```
Enter username: Snehanjali
Enter password: sony0610
User registered successfully!
PS C:\Users\uppus\Downloads\lab_20.3>
```

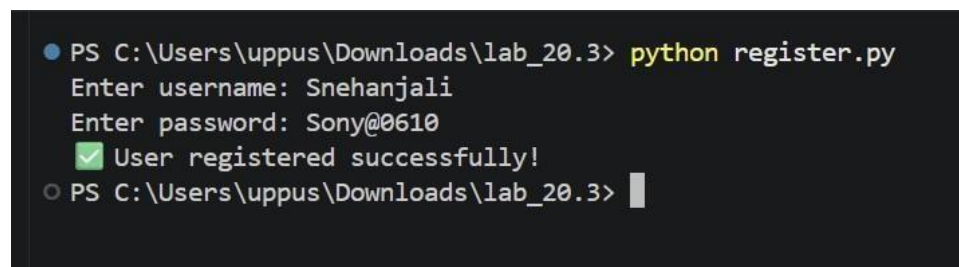
- The program accepts passwords without any validation checks.
- Weak passwords are allowed, making the system insecure.

Corrected Code:



```
1 import re
2 def is_strong_password(password):
3     if len(password) < 8:
4         print("❌ Minimum 8 characters required")
5         return False
6     if not re.search(r"[A-Z]", password):
7         print("❌ Must include uppercase letter")
8         return False
9     if not re.search(r"[a-z]", password):
10        print("❌ Must include lowercase letter")
11        return False
12    if not re.search(r"[0-9]", password):
13        print("❌ Must include number")
14        return False
15    if not re.search(r"[!@#%$^&*]", password):
16        print("❌ Must include special character")
17        return False
18    return True
19
20 username = input("Enter username: ")
21 password = input("Enter password: ")
22
23 if is_strong_password(password):
24     print("✅ User registered successfully!")
25 else:
26     print("❌ Weak password! Try again.")
```

Output:



```
PS C:\Users\uppus\Downloads\lab_20.3> python register.py
Enter username: Snehanjali
Enter password: Sony@0610
✅ User registered successfully!
PS C:\Users\uppus\Downloads\lab_20.3>
```

Explanation:

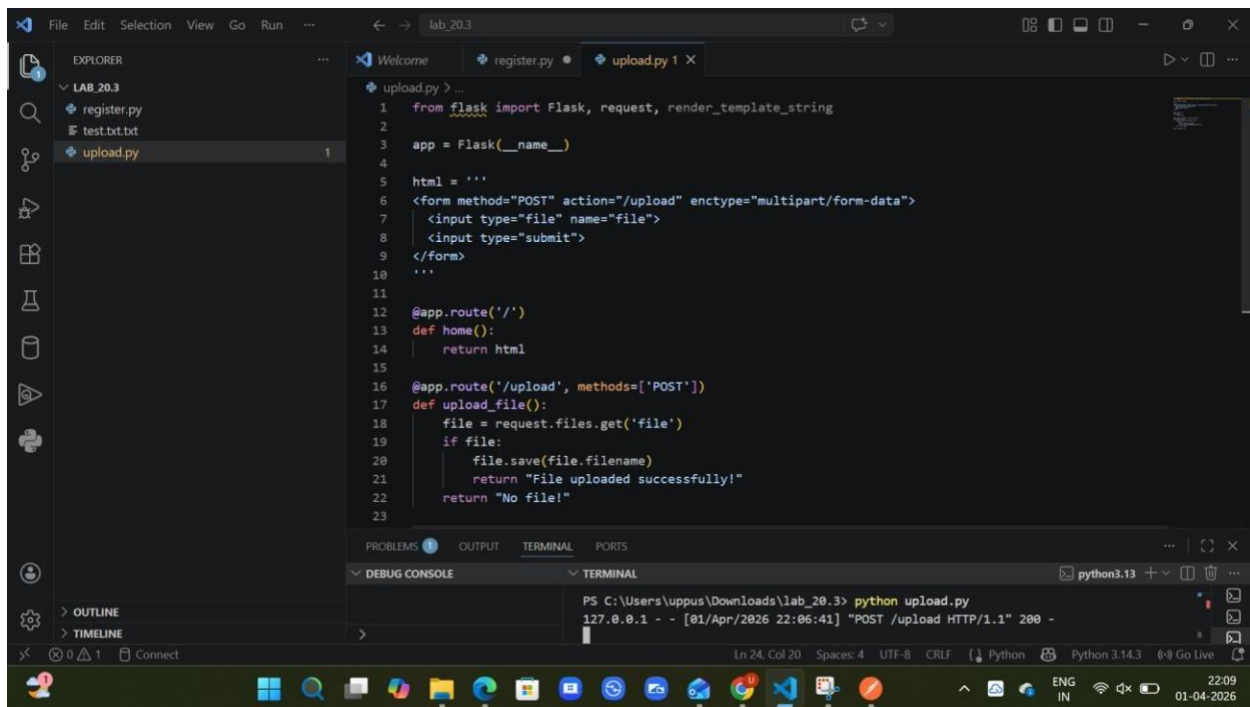
The initial program does not check the strength of the password. It accepts any password entered by the user, including weak ones. This makes the system insecure and vulnerable to attacks.

Task2: Insecure File Upload Handling

Prompt:

Generate a Flask-based file upload program that allows users to upload files.

Code: Ai Generated Code



```
1 from flask import Flask, request, render_template_string
2
3 app = Flask(__name__)
4
5 html = '''
6 <form method="POST" action="/upload" enctype="multipart/form-data">
7   <input type="file" name="file">
8   <input type="submit">
9 </form>
10 '''
11
12 @app.route('/')
13 def home():
14     return html
15
16 @app.route('/upload', methods=['POST'])
17 def upload_file():
18     file = request.files.get('file')
19     if file:
20         file.save(file.filename)
21         return "File uploaded successfully!"
22     return "No file!"
23
```

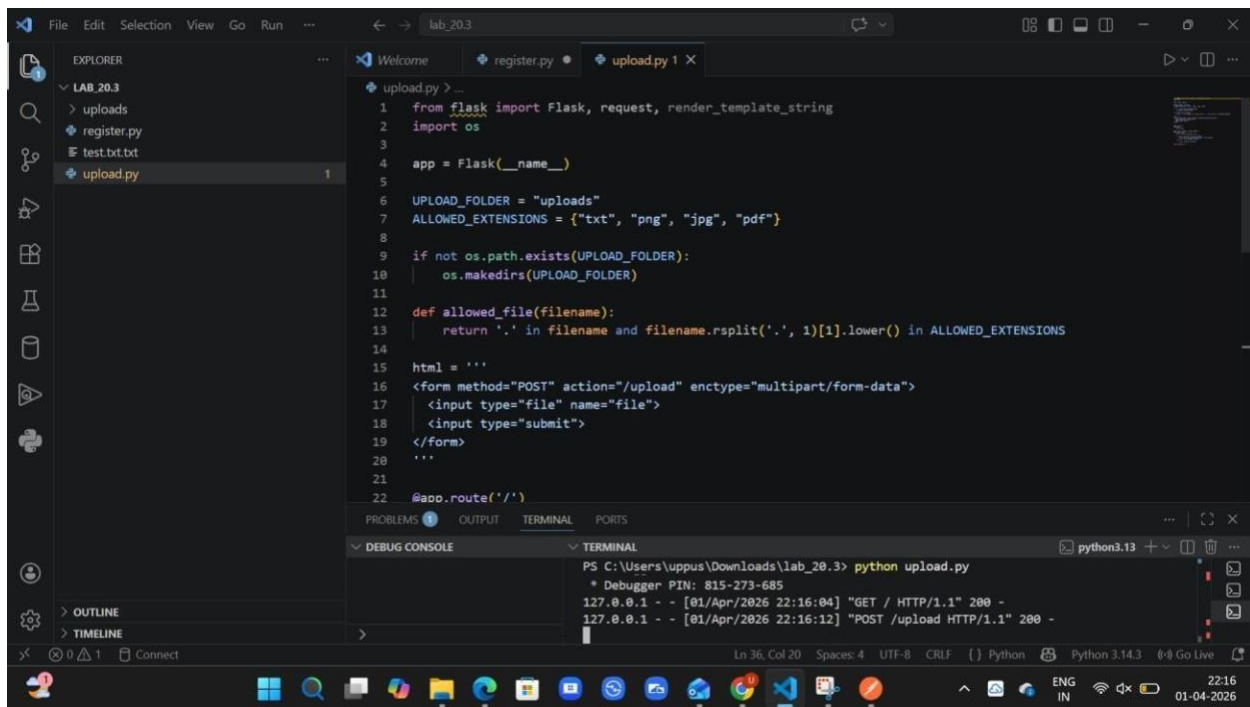
PS C:\Users\uppus\Downloads\lab_20.3> python upload.py
127.0.0.1 - - [01/Apr/2026 22:06:41] "POST /upload HTTP/1.1" 200 -

Output:



- The program allows uploading any type of file
- No validation of file extension

Corrected code:

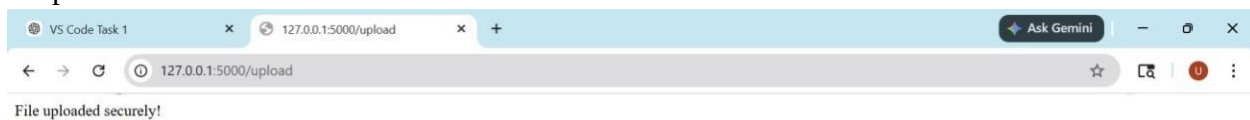


```
1 from flask import Flask, request, render_template_string
2 import os
3
4 app = Flask(__name__)
5
6 UPLOAD_FOLDER = "uploads"
7 ALLOWED_EXTENSIONS = {"txt", "png", "jpg", "pdf"}
8
9 if not os.path.exists(UPLOAD_FOLDER):
10     os.makedirs(UPLOAD_FOLDER)
11
12 def allowed_file(filename):
13     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
14
15 html = '''
16 <form method="POST" action="/upload" enctype="multipart/form-data">
17   <input type="file" name="file">
18   <input type="submit">
19 </form>
20 '''
21
22 @app.route('/')
23 def index():
24     return render_template_string(html)
25
26 if __name__ == '__main__':
27     app.run(debug=True)
```

Terminal Output:

```
PS C:\Users\uppus\Downloads\lab_20.3> python upload.py
* Debugger PIN: 815-273-685
127.0.0.1 - - [01/Apr/2026 22:16:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:16:12] "POST /upload HTTP/1.1" 200 -
```

Output:



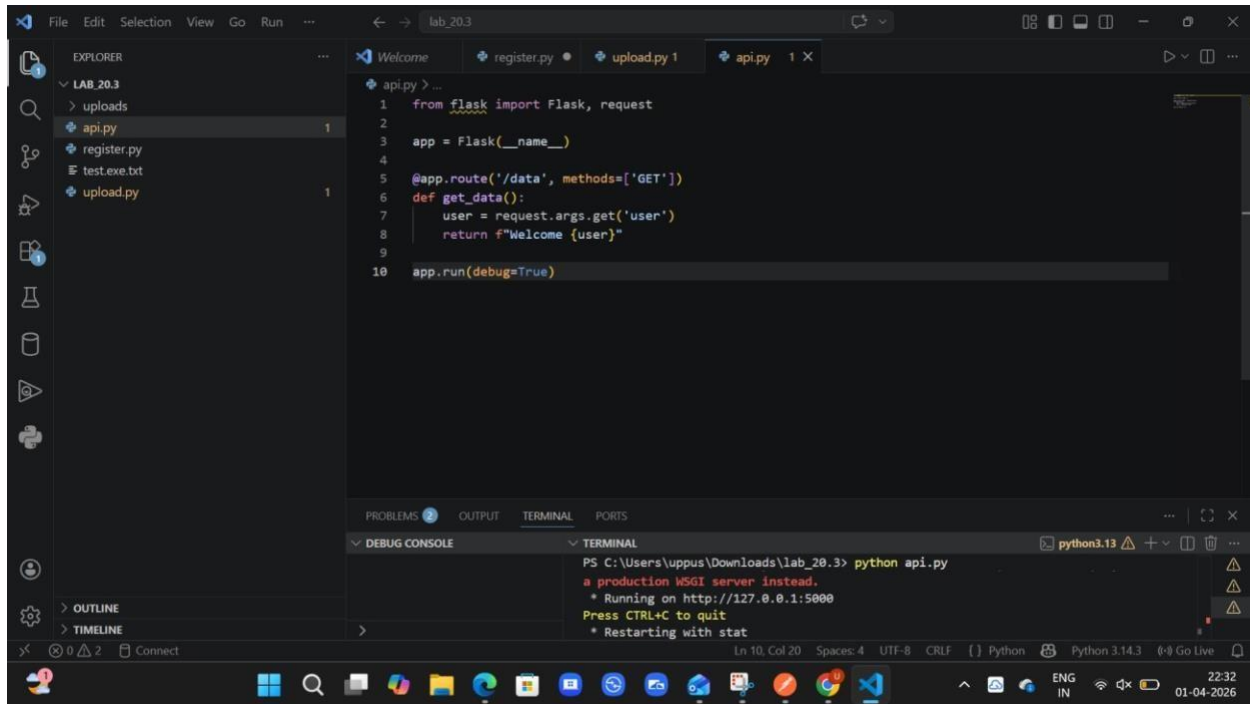
Explanation:

- The initial code allows uploading any file without checking its type, making it insecure.
- Malicious files like .exe can be uploaded, which may harm the system.

- The improved code restricts file types and allows only safe formats like .txt, .png, .jpg, and .pdf. Task3: API Security Review

Prompt:

Generate a Python Flask API endpoint for user data Code:

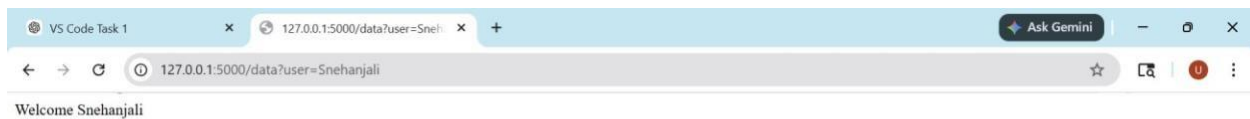


The screenshot shows a Visual Studio Code editor window with a project named 'lab_20.3'. The Explorer pane on the left shows a file structure with 'uploads' and 'api.py'. The main editor area displays the code for 'api.py', which includes a Flask application with a single endpoint '/data' that returns a welcome message. The terminal at the bottom shows the command 'python api.py' being executed, resulting in a warning about a production WSGI server and the application running on http://127.0.0.1:5000.

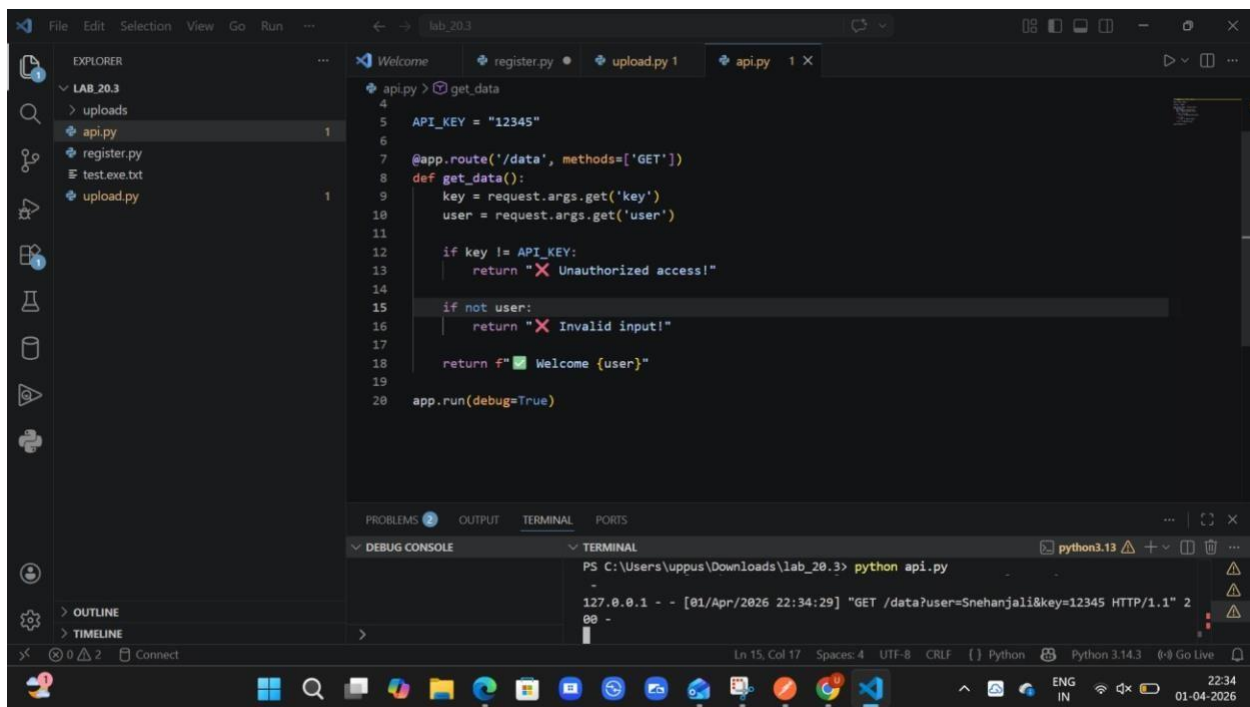
```
1 from flask import Flask, request
2
3 app = Flask(__name__)
4
5 @app.route('/data', methods=['GET'])
6 def get_data():
7     user = request.args.get('user')
8     return f"Welcome {user}"
9
10 app.run(debug=True)
```

PS C:\Users\uppus\Downloads\lab_20.3> python api.py
a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat

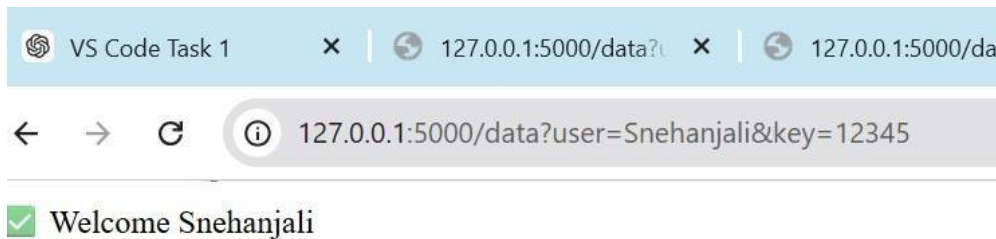
Output:



Corrected code:



Output:



Explanation:

- The initial API does not have authentication, allowing anyone to access it
- It does not validate user input, making it insecure
- The improved API uses an API key and input validation for secure access

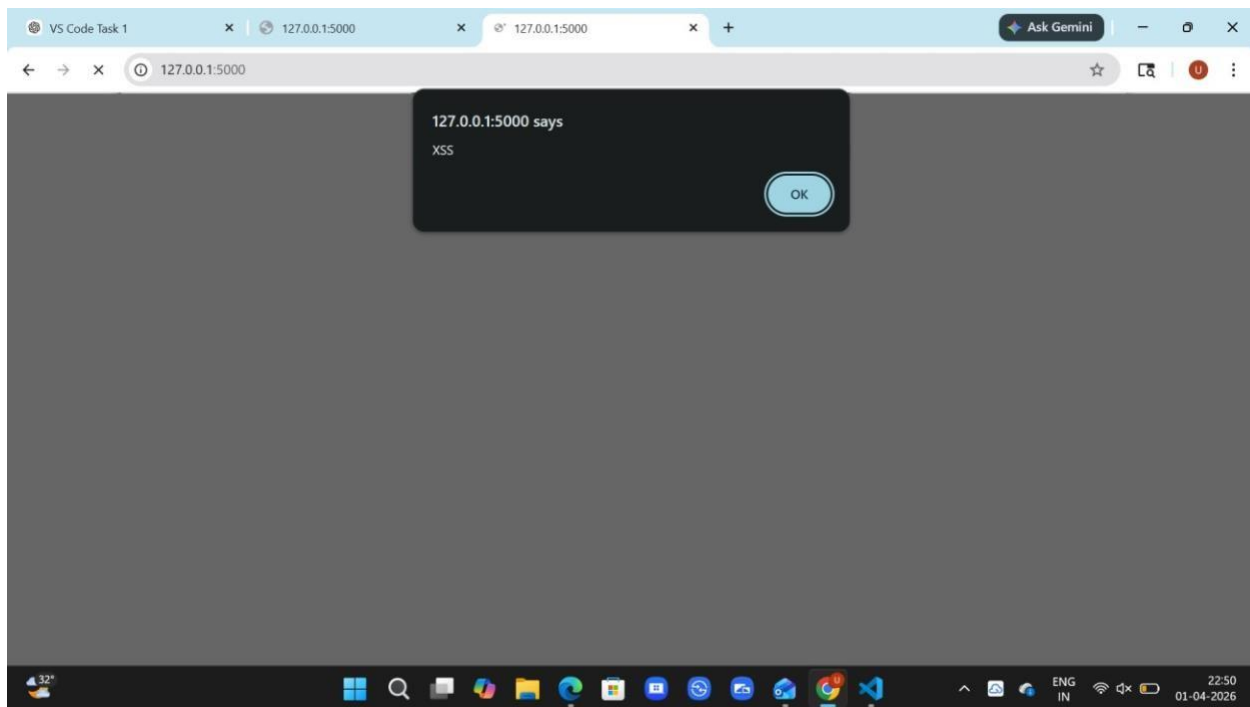
Task 4: Cross-Site Scripting (XSS) Check

Prompt: Generate a feedback form using HTML and JavaScript

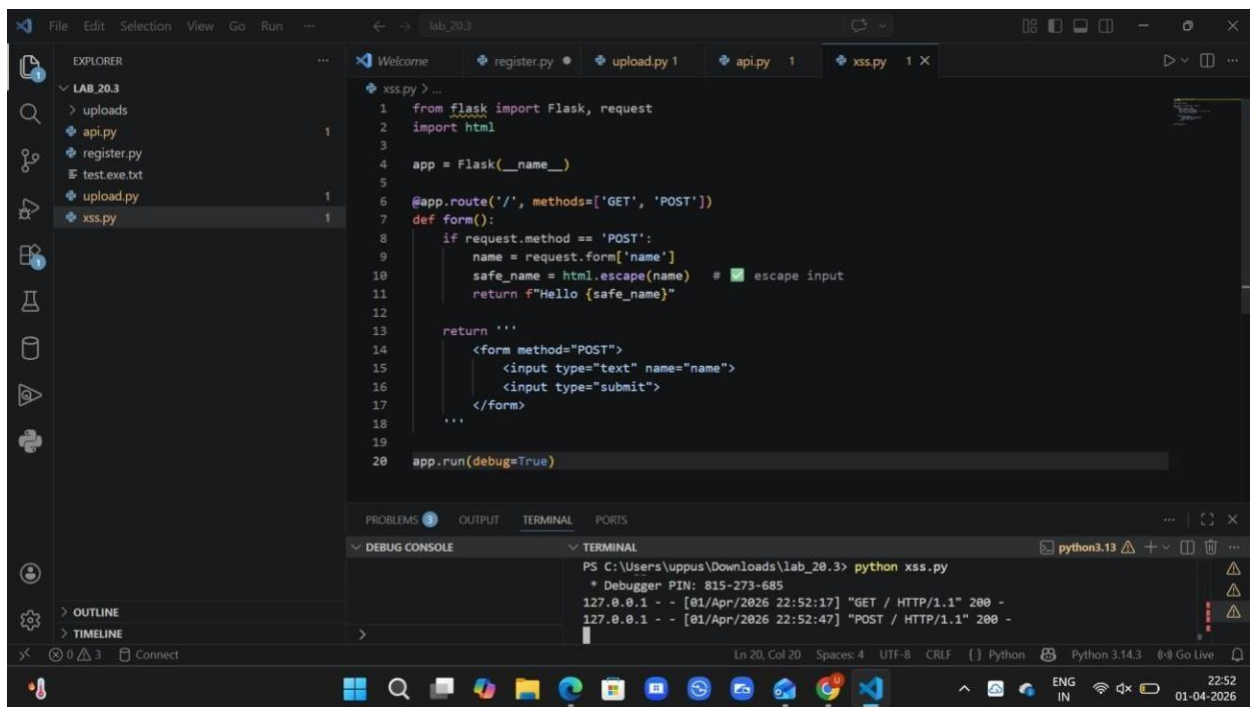
Code:

A screenshot of a VS Code editor window. The Explorer panel on the left shows a project named 'LAB_203' with files 'uploads', 'api.py', 'register.py', 'test.exe.txt', 'upload.py', and 'xss.py'. The main editor area shows the code for 'xss.py'. The code is a Flask application that defines a route '/' with methods 'GET' and 'POST'. The 'POST' method calls a function 'form()' which returns a string 'Hello {name}'. The 'GET' method returns an HTML form with a text input field named 'name' and a submit button. The application is run with debug=True. The terminal panel at the bottom shows the command 'python xss.py' and the output of the application, which is a POST request and a GET request, both returning a 200 status code.

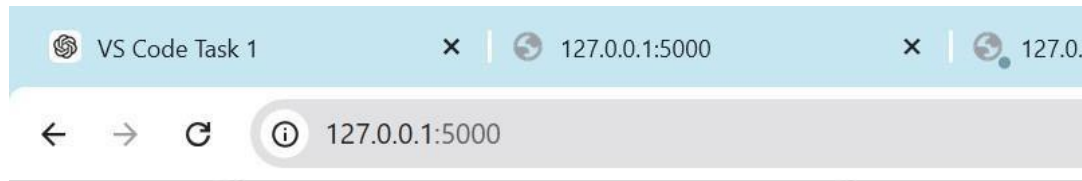
Output:



Corrected Code:



Output:



Hello Hello `<script>alert('XSS')</script>`

Explanation:

- The initial code displays user input directly without any validation, making it vulnerable to XSS attacks.
- Malicious scripts like `<script>alert('XSS')</script>` can execute in the browser.
- The secure code uses input escaping to prevent script execution and ensures safe output.

Task 5: SQL Injection Prevention

Prompt: Generate a Python script using SQLite to fetch user details from a database.

Code:

```
1 import sqlite3
2
3 conn = sqlite3.connect("users.db")
4 cursor = conn.cursor()
5
6 username = input("Enter username: ")
7
8 query = f"SELECT * FROM users WHERE username = '{username}'"
9 cursor.execute(query)
10
11 result = cursor.fetchall()
12 print(result)
```

Terminal Output:

```
[[('Sneha'), (2, 'Admin')]]
PS C:\Users\uppus\Downloads\lab_20.3>
```

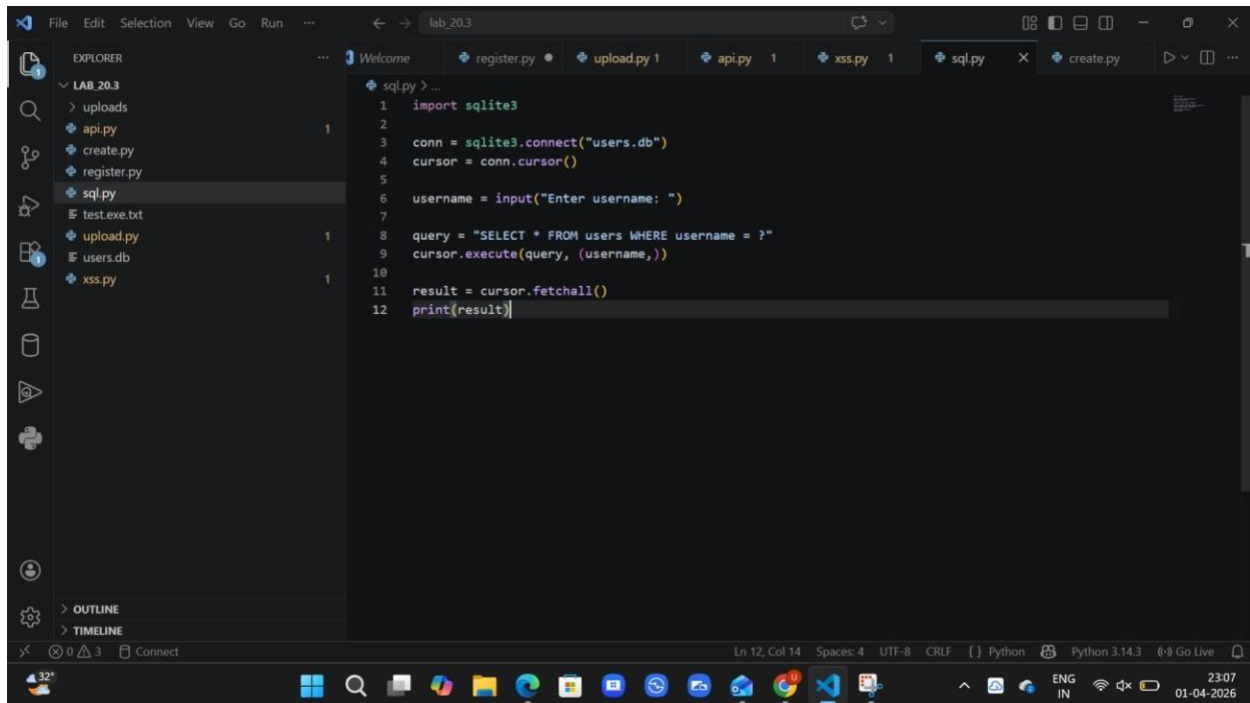
Output:

```

PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[(1, 'Sneha'), (2, 'Admin')]
PS C:\Users\uppus\Downloads\lab_20.3>

```

Corrected code:



```

1 import sqlite3
2
3 conn = sqlite3.connect("users.db")
4 cursor = conn.cursor()
5
6 username = input("Enter username: ")
7
8 query = "SELECT * FROM users WHERE username = ?"
9 cursor.execute(query, (username,))
10
11 result = cursor.fetchall()
12 print(result)

```

Output:

```

PS C:\Users\uppus\Downloads\lab_20.3> python sql.py
Enter username: ' OR '1'='1
[]
PS C:\Users\uppus\Downloads\lab_20.3>

```

Explanation:

- The secure code uses parameterized queries (?) instead of string concatenation.
- User input is treated as data, not as part of the SQL query.
- This prevents SQL Injection and ensures safe database operations.

